Facultad de Ingeniería en Sistemas, Universidad Mariano Gálvez de Guatemala

Ingeniería en Sistemas de Información y Ciencias de la Computación

Aplicación de la trigonometría vectorial en el desarrollo de un motor de campo de visión 2D para la simulación de iluminación en tiempo real

Jose Carlos Alfaro García

Carné: 0906-26-9140

Curso: Precálculo

Docente: M.A. Ing. en Sistemas Claudio Orazio López Calderón

31 de octubre de 2026

Introducción

Hoy en día casi cualquier videojuego, simulador o aplicación gráfica necesita decidir qué puede ver un personaje y qué queda oculto detrás de un obstáculo. Ese cálculo de zonas visibles, conocido en inglés como *Field of View* (FOV), parece simple desde el lado del usuario, pero detrás hay bastante matemática. En este proyecto se desarrolló un simulador de línea de visión en dos dimensiones que corre en el navegador y que permite ver, en tiempo real, cómo la luz de una linterna se bloquea al chocar con las paredes.

Para lograrlo se aplicaron temas vistos en el curso de Precálculo: los catetos y la hipotenusa de un triángulo rectángulo, las funciones seno y coseno para descomponer un vector, la conversión entre grados y radianes, y la función arcotangente en su variante `atan2`, que sirve para calcular hacia dónde apunta el jugador a partir de la posición del mouse. Además, se usó una comprobación de colisiones con rectángulos alineados a los ejes. El programa está escrito en JavaScript y se dibuja sobre un elemento Canvas de HTML5.

Descripción del problema

Cuando se programa un juego o simulador en 2D, una de las primeras preguntas es qué zonas del mapa puede ver el jugador. Una forma de resolverlo sería calcular la intersección exacta entre cada rayo de luz y cada lado de cada obstáculo, pero la cantidad de operaciones crece rápido conforme se agregan objetos a la escena, y repetirlo 60 veces por segundo exige que el cálculo sea liviano.

La otra opción es dibujar la luz sin ninguna regla matemática. En ese caso el cono de visión atraviesa las paredes y la simulación pierde sentido, porque el jugador vería cosas que en teoría están tapadas. Lo que se necesita es un mecanismo que lance varios rayos dentro de un abanico de ángulos, que parta de la posición del jugador y que se detenga en el punto donde el rayo toca una superficie sólida.

Justificación

Este proyecto muestra que la trigonometría que se estudia en el curso tiene un uso directo en el trabajo de un ingeniero en sistemas. Un problema que a simple vista es visual, como iluminar una habitación dentro de un juego, termina resolviéndose con razones trigonométricas y operaciones con vectores.

La técnica de raycasting divide el espacio continuo en pasos pequeños que la computadora puede revisar uno por uno. Como las funciones seno, coseno y `atan2` ya vienen incluidas en JavaScript, en cada fotograma se pueden proyectar más de cien rayos sin exigirle demasiado al equipo (Akenine-Möller et al., 2018). Por esta razón se eligió el tema: permite practicar matemática clásica en un programa que funciona y que además sirve de base para temas posteriores, como la detección de colisiones y los gráficos por computadora.

Objetivos

Objetivo general

Desarrollar un algoritmo que calcule el campo de visión en 2D usando razones trigonométricas, geometría analítica y proyección de vectores, para simular visibilidad e iluminación en tiempo real dentro de una aplicación interactiva.

Objetivos específicos

1. Utilizar la función arcotangente 2 (`atan2`) para obtener el ángulo de orientación del jugador a partir de la posición del mouse.
2. Proyectar cada rayo paso a paso (*ray marching*), usando seno y coseno para repartir la distancia recorrida entre los ejes X y Y.
3. Programar la solución en JavaScript con el elemento Canvas de HTML5 y comprobar que el resultado concuerda con la teoría a 60 fotogramas por segundo.

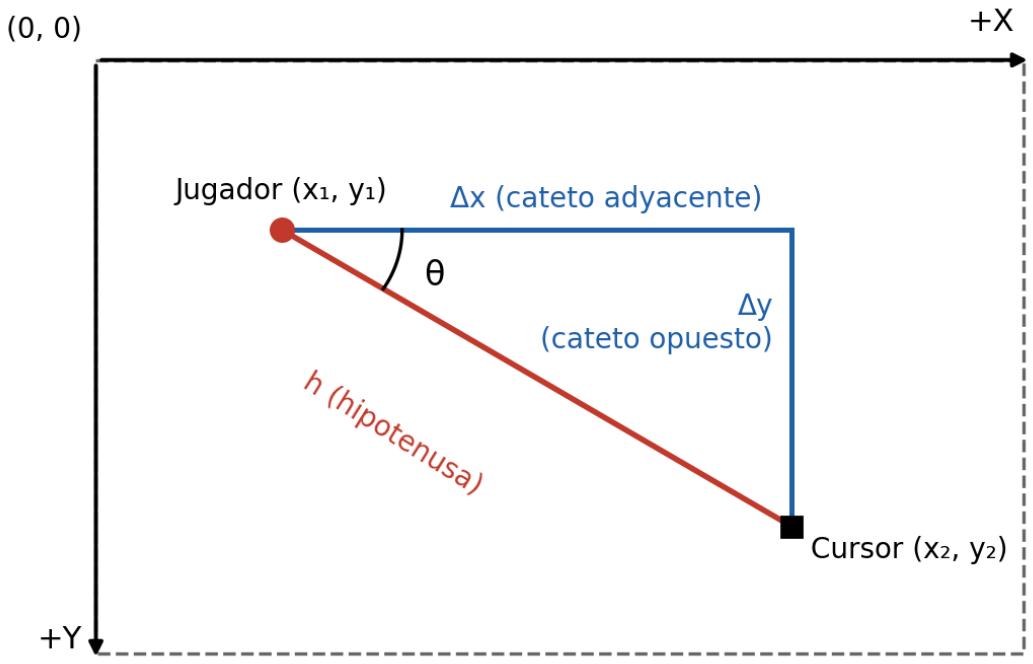
Teoría y conceptos

Sistema de coordenadas en pantalla

A diferencia del plano cartesiano que se usa en clase, el lienzo (*canvas*) coloca su origen (0, 0) en la esquina superior izquierda. La coordenada X aumenta hacia la derecha, pero la coordenada Y aumenta hacia abajo. Esto cambia la forma de interpretar los ángulos: en pantalla, un ángulo positivo gira en el sentido de las manecillas del reloj. La Figura 1 muestra este sistema junto con los elementos del triángulo rectángulo que se explican en el siguiente apartado.

Figura 1

Sistema de coordenadas del canvas y triángulo rectángulo formado por el jugador y el cursor



Nota. Elaboración propia. El origen se encuentra en la esquina superior izquierda y el eje Y crece hacia abajo.

Catetos y distancia euclidiana

Para relacionar la posición del jugador (x_1, y_1) con la del cursor (x_2, y_2) se calcula la diferencia en cada eje. Estas diferencias son los catetos de un triángulo rectángulo (Stewart et al., 2017):

$$\Delta x = x_2 - x_1 \qquad \text{(cateto adyacente)}$$

$$\Delta y = y_2 - y_1 \qquad \text{(cateto opuesto)}$$

Con el teorema de Pitágoras, la distancia en línea recta entre ambos puntos, que es la hipotenusa h, se obtiene así:

$$h = \sqrt{[(\Delta x)^2 + (\Delta y)^2]}$$

Arcotangente 2 y ángulo de orientación

La tangente del ángulo relaciona ambos catetos:

$$\tan(\theta) = \Delta y / \Delta x$$

Para despejar θ se usa la función inversa, la arcotangente. Sin embargo, calcular arctan(Δy/Δx) tiene dos problemas: no se puede resolver cuando Δx = 0 y no distingue entre cuadrantes, porque dos puntos en direcciones opuestas dan el mismo cociente. Por eso en programación se utiliza `Math.atan2(dy, dx)`, que recibe los dos catetos por separado, evita la división y devuelve un ángulo entre −π y +π radianes que ya considera el cuadrante correcto (Mozilla Developer Network, s. f.-b).

$$\theta_{rad} = \text{atan2}(\Delta y, \Delta x)$$

Conversión entre grados y radianes

Las funciones trigonométricas de JavaScript trabajan en radianes, pero los grados sexagesimales (0° a 360°) son más fáciles de interpretar cuando se revisa el programa. Como π rad = 180°, las conversiones son las siguientes:

$$\theta_{grados} = \theta_{rad} \times 180 / \pi \qquad \text{(radianes a grados)}$$

$$\theta_{rad} = \theta_{grados} \times \pi / 180 \qquad \text{(grados a radianes)}$$

Descomposición vectorial con seno y coseno

Un vector de longitud uno con inclinación θ tiene componente horizontal cos(θ) y componente vertical sen(θ). Si ese vector se desplaza una distancia Δs, que en el programa es el tamaño del paso, el nuevo punto se calcula con:

$$x_{nuevo} = x_{actual} + \cos(\theta) \cdot \Delta s$$

$$y_{nuevo} = y_{actual} + \text{sen}(\theta) \cdot \Delta s$$

Detección de colisiones AABB

Cada pared es un rectángulo definido por su esquina superior izquierda (x_p, y_p), su ancho W y su alto H. Un punto (x, y) choca con ella solo si cumple las cuatro condiciones al mismo tiempo. Este método se conoce como AABB (*Axis-Aligned Bounding Box*), porque los lados del rectángulo son paralelos a los ejes:

$$\text{Colisión} = (x \geq x_p) \wedge (x \leq x_p + W) \wedge (y \geq y_p) \wedge (y \leq y_p + H)$$

Desarrollo del tema

Arquitectura del sistema

El programa se organiza en cuatro partes:

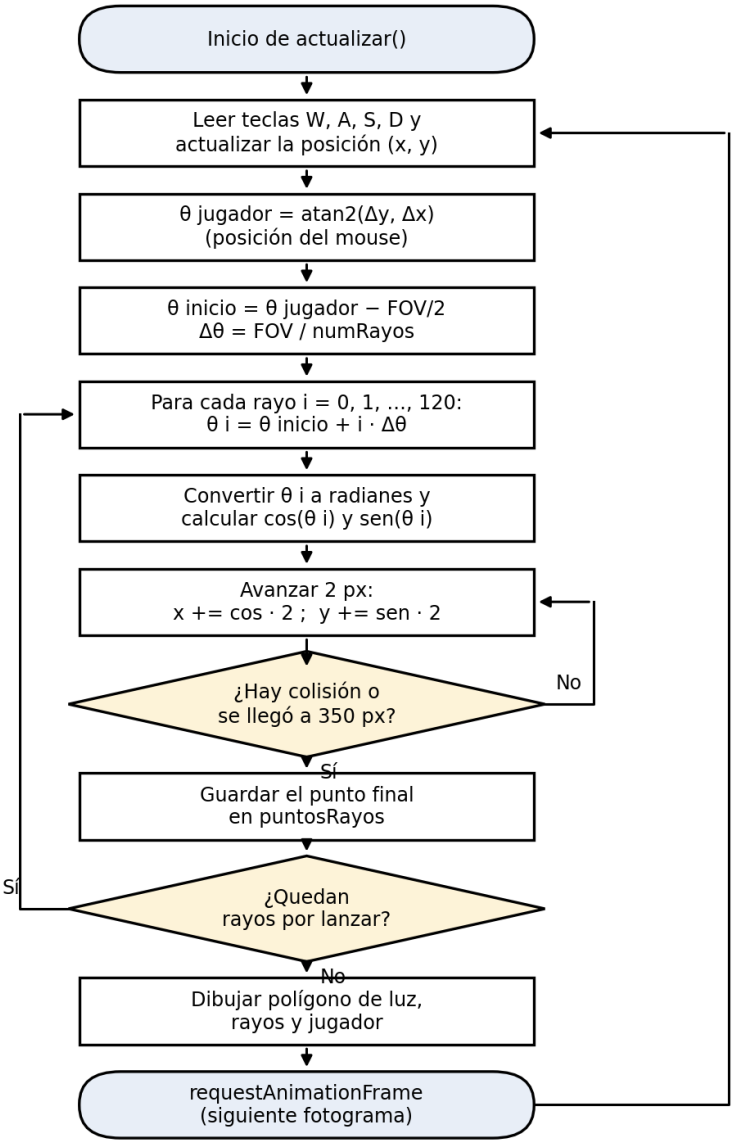
- **Estado del programa.** Guarda las propiedades del jugador (posición, ángulo y campo de visión) y la geometría de las paredes del mapa.
- **Entrada del usuario.** Captura la posición del mouse y las teclas W, A, S y D.
- **Núcleo matemático.** Ejecuta los ciclos de proyección de cada rayo con los incrementos trigonométricos.
- **Renderizado.** Dibuja en el canvas las paredes, el polígono de visibilidad y al jugador.

Flujo del algoritmo

La función principal, `actualizar()`, se ejecuta en cada fotograma y sigue la secuencia que se resume en la Figura 2 y se detalla a continuación.

Figura 2

Diagrama de flujo del ciclo principal del motor de raycasting



Nota. Elaboración propia a partir del código fuente del proyecto.

1. **Lectura del teclado.** Si se presiona W, A, S o D, la posición (x, y) del jugador cambia 3 píxeles en la dirección correspondiente.

2. **Cálculo de la orientación.** Cuando el mouse se mueve, se obtiene el ángulo hacia el puntero con `Math.atan2()` y se convierte a grados.
3. **Generación del abanico de visión.**
    - Se define la apertura del cono, FOV = 60°.
    - Se obtiene el límite izquierdo del cono: $\theta_{inicio} = \theta_{jugador} - FOV/2$.
    - Se calcula el incremento angular por rayo: $\Delta\theta = FOV / numRayos$, que con 120 rayos equivale a 0,5° entre un rayo y el siguiente.
4. **Proyección de cada rayo (ray marching).**
    - Para cada rayo i, de 0 a 120, se calcula su ángulo: $\theta_i = \theta_{inicio} + i \cdot \Delta\theta$.
    - Ese ángulo se convierte a radianes y se obtienen cos(θ) y sen(θ).
    - El rayo avanza de 2 en 2 píxeles, hasta un alcance máximo de 350 píxeles.
    - En cada avance se consulta `hayColision(x, y)`. Si el punto entra en un rectángulo o sale del lienzo, el rayo se detiene.
    - El punto final se guarda en el arreglo `puntosRayos`.
5. **Dibujo de la luz.** Con `beginPath`, `moveTo` y `lineTo` se traza un polígono que une la posición del jugador con todos los puntos guardados en `puntosRayos`, y se rellena con un amarillo translúcido. Después se dibujan los rayos individuales y al jugador.

Código fuente del proyecto

A continuación se presenta la implementación completa en HTML5 y JavaScript. Para ejecutarla basta con guardar el contenido en un archivo con extensión .html y abrirlo en cualquier navegador.

URL GitHub Pages: https://joca-alfaro.github.io/Proyecto-Pre.Calculo/

URL repositorio GitHub: https://github.com/Joca-Alfaro/Proyecto-Pre.Calculo

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Motor 2D - Trigonometría</title>
    <style>
        body { margin: 0; background-color: #1a1a1a; color: white; font-family: sans-serif;
display: flex; flex-direction: column; align-items: center; }
        canvas { border: 2px solid #444; background: #050505; margin-top: 10px; cursor:
crosshair; }
        .info { margin-top: 10px; width: 800px; display: flex; justify-content: space-between;
font-size: 14px; }
    </style>
</head>
<body>
    <h2>Simulador de Línea de Visión (CAMPO DE VISIÓN 2D)</h2>
    <canvas id="CajaCanvas" width="800" height="600"></canvas>
    <div class="info">
        <span>Controles: Mueve el mouse para apuntar | WASD para moverte</span>
        <span id="estado">Calculando...</span>
    </div>

<script>
const CajaCanvasJs = document.getElementById('CajaCanvas');
const ctx = CajaCanvasJs.getContext('2d');

// Configuración del Jugador
const jugador = {
    x: 400,
    y: 300,
    angulo: 0,          // Ángulo en grados
    campoVision: 200,    // Apertura del cono de visión (FOV en grados)
    numRayos: 120,      // Cantidad de rayos trigonométricos
    distMax: 350        // Alcance máximo de la linterna en píxeles
};

// Obstáculos en el mundo (paredes)
const paredes = [
    { x: 150, y: 100, ancho: 100, alto: 150 },
    { x: 500, y: 150, ancho: 150, alto: 80 },
    { x: 300, y: 400, ancho: 200, alto: 100 }
];

// Detección de movimiento por teclado
const teclas = {};
window.addEventListener('keydown', e => teclas[e.key.toLowerCase()] = true);
window.addEventListener('keyup', e => teclas[e.key.toLowerCase()] = false);

// Apuntado con el mouse mediante Arcotangente (Trigonometría Inversa)
CajaCanvasJs.addEventListener('mousemove', e => {
```

```
    const rectangulo = CajaCanvasJs.getBoundingClientRect();
    const mouseX = e.clientX - rectangulo.left;
    const mouseY = e.clientY - rectangulo.top;


    // Cálculo de catetos hacia la posición del mouse
    const distanciaX = mouseX - jugador.x;
    const distanciaY = mouseY - jugador.y;


    // Conversión de arcotangente2 (radianes a grados)
    jugador.angulo = Math.atan2(distanciaY, distanciaX) * (180 / Math.PI);
});

// Función de conversión de grados sexagesimales a radianes
function gradosARadianes(grados) {
    return grados * (Math.PI / 180);
}

// Comprobación de colisión entre un punto y las paredes del mundo
function hayColision(x, y) {
    if (x < 0 || x > CajaCanvasJs.width || y < 0 || y > CajaCanvasJs.height) return true;
    for (let pared of paredes) {
        if (x >= pared.x && x <= pared.x + pared.ancho && y >= pared.y && y <= pared.y +
pared.alto) {
            return true;
        }
    }
    return false;
}

// Ciclo Principal de Renderizado
function actualizar() {
    // Movimiento básico con teclas WASD
    if (teclas['w']) jugador.y -= 3;
    if (teclas['s']) jugador.y += 3;
    if (teclas['a']) jugador.x -= 3;
    if (teclas['d']) jugador.x += 3;


    // Limpieza de pantalla
    ctx.clearRect(0, 0, CajaCanvasJs.width, CajaCanvasJs.height);


    // Dibujar obstáculos (paredes)
    ctx.fillStyle = '#334455';
    paredes.forEach(pared => ctx.fillRect(pared.x, pared.y, pared.ancho, pared.alto));


    // LÓGICA DE RAYCASTING TRIGONOMÉTRICO
    const puntosRayos = [];
    const mitadFOV = jugador.campoVision / 2;
    const anguloInicio = jugador.angulo - mitadFOV;
    const pasoAngulo = jugador.campoVision / jugador.numRayos;


    for (let i = 0; i <= jugador.numRayos; i++) {
        const anguloActualGrados = anguloInicio + (i * pasoAngulo);
        const radianes = gradosARadianes(anguloActualGrados);


        // Proyección incremental mediante Catetos: cos(rad), sen(rad)
        const coseno = Math.cos(radianes);
        const seno = Math.sin(radianes);


        let rayoX = jugador.x;
        let rayoY = jugador.y;
        let distancia = 0;
        const pasoTamano = 2;


        while (distancia < jugador.distMax) {
            rayoX += coseno * pasoTamano;
```

```
            rayoY += seno * pasoTamano;
            distancia += pasoTamano;

            if (hayColision(rayoX, rayoY)) break;
        }
        puntosRayos.push({ x: rayoX, y: rayoY });
    }

    // Dibujar el Polígono de Luz (Campo de Visión)
    ctx.beginPath();
    ctx.moveTo(jugador.x, jugador.y);
    puntosRayos.forEach(punto => ctx.lineTo(punto.x, punto.y));
    ctx.closePath();
    ctx.fillStyle = 'rgba(255, 230, 150, 0.35)';
    ctx.fill();

    // Dibujar Líneas de Rayos individuales
    ctx.strokeStyle = 'rgba(255, 200, 50, 0.15)';
    ctx.beginPath();
    puntosRayos.forEach(punto => {
        ctx.moveTo(jugador.x, jugador.y);
        ctx.lineTo(punto.x, punto.y);
    });
    ctx.stroke();

    // Dibujar Jugador
    ctx.fillStyle = '#ff4444';
    ctx.beginPath();
    ctx.arc(jugador.x, jugador.y, 8, 0, Math.PI * 2);
    ctx.fill();

    // Actualizar texto informativo de estado
    document.getElementById('estado').innerText =
        `Pos: (${Math.round(jugador.x)}, ${Math.round(jugador.y)}) | Ángulo:
${Math.round(jugador.angulo)}°`;

    requestAnimationFrame(actualizar);
}

actualizar();
</script>
</body>
</html>
```

Conclusiones

1. Con el proyecto se comprobó que el seno, el coseno y la tangente sirven para resolver un problema práctico de proyección en gráficos por computadora. Descomponer cada rayo en sus componentes cos(θ) y sen(θ) fue lo que permitió avanzarlo en cualquier dirección del plano.
2. El uso de `Math.atan2()` permitió calcular el ángulo del jugador hacia el mouse en los 360°, sin errores por división entre cero y sin tener que revisar a mano en qué cuadrante estaba el cursor.
3. La técnica de *ray marching* fue suficiente para este caso: al avanzar los rayos en pasos de 2 píxeles y revisar las colisiones con la prueba AABB, se detectaron las paredes sin necesidad de calcular intersecciones exactas entre segmentos.
4. El trabajo también mostró que los temas de Precálculo no son solo teoría. Al llevarlos al código se nota de forma directa que, por ejemplo, una conversión mal hecha entre grados y radianes cambiaría por completo lo que aparece en pantalla.

Recomendaciones

1. **Intersección analítica de segmentos.** Si el mapa llegara a tener muchos obstáculos, conviene reemplazar el avance de 2 en 2 píxeles por el cálculo directo de la intersección entre el rayo y cada lado de las paredes, usando determinantes y productos escalar y vectorial. Así se reducen las vueltas del ciclo `while` y se evita que un rayo atraviese por error una pared muy delgada, algo que con pasos de 2 píxeles sí podría ocurrir.
2. **Partición del espacio.** En mapas grandes se aconseja usar estructuras como un *QuadTree* o una rejilla espacial, de modo que `hayColision()` revise solo los obstáculos cercanos al jugador y no todos los del mapa.
3. **Radianes en el estado interno.** Para evitar conversiones repetidas con `gradosARadianes()`, se sugiere guardar la rotación de las entidades directamente en radianes dentro del estado del juego.

Referencias

Akenine-Möller, T., Haines, E., & Hoffman, N. (2018). *Real-time rendering* (4.ª ed.). CRC Press.

Mozilla Developer Network. (s. f.-a). *Canvas API*. MDN Web Docs. https://developer.mozilla.org/es/docs/Web/API/Canvas_API

Mozilla Developer Network. (s. f.-b). *Math.atan2()*. MDN Web Docs. https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Math/atan2

Stewart, J., Redlin, L., & Watson, S. (2017). *Precálculo: Matemáticas para el cálculo* (7.ª ed.). Cengage Learning.